CHAPTER 16

Template Method Pattern

This chapter covers the Template Method pattern.

GoF Definition

Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. The Template Method pattern lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

Concept

With the Template Method pattern, you define the minimum or essential structure of an algorithm. Then you defer some responsibility to the subclasses. The key idea is that you can redefine certain steps of an algorithm, but those changes will not impact the core architecture.

This design pattern is useful if you have to implement a multistep algorithm and you want to allow customization through subclasses.

Real-Life Example

Suppose you want to make a pizza. You use the basic mechanism each time you make a pizza, but you include the final materials based on your customer's choices. For example, the customer may want a vegetarian pizza or a meat pizza. A customer can even opt for different toppings such as bacon, onions, extra cheese, mushrooms, and so on.

© Vaskaran Sarcar 2018
V. Sarcar, *Design Patterns in C#*, https://doi.org/10.1007/978-1-4842-3640-6_16

www.EBooksWorld.ir

Computer World Example

Suppose you have been hired to design an online engineering degree course. You know that, in general, the first semester of the course will be the same for all courses. For subsequent semesters, you need to add new papers or subjects to the application based on the course opted by a student. The Template Method pattern makes sense when you want to avoid duplicate code in your application but allow subclasses to change some specific details of the base class workflow to bring varying behavior to the application. (However, you may not want to override the base methods entirely to make radical changes in the subclasses. In this way, the pattern differs from simple polymorphism.)

Illustration

Assume that each engineering student needs to pass mathematics and also demonstrate soft skills (such as communication skills, people management skills, and so on) in their initial semesters to obtain their degrees. Later you will add some special papers to their courses based on their chosen paths (computer science or electronics).

www.EBooksWorld.ir

Class Diagram

Figure 16-1 shows the class diagram.

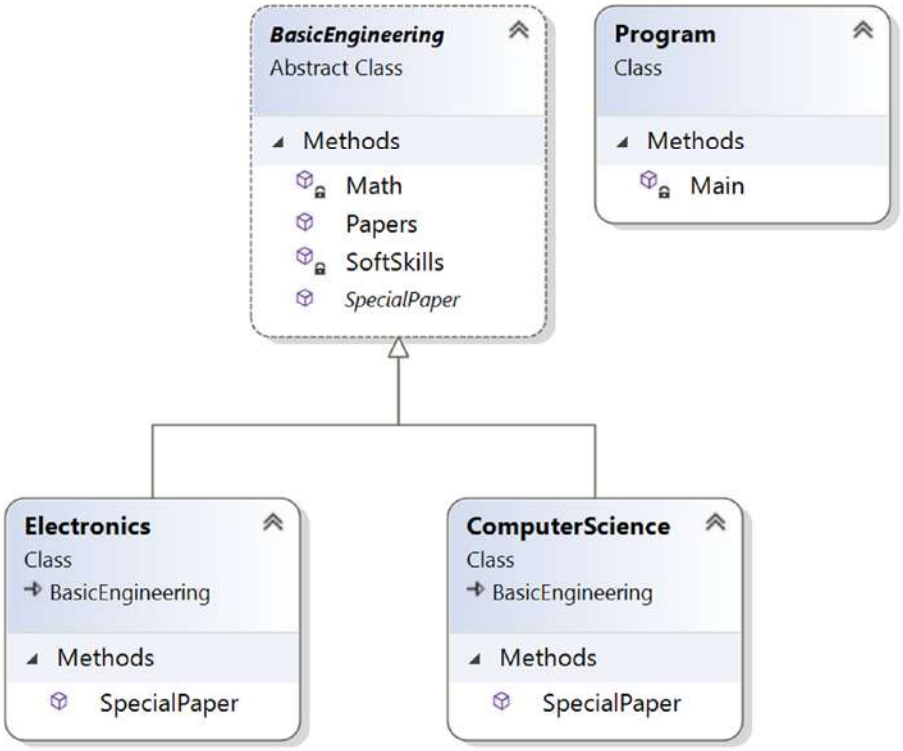


Figure 16-1. *Class diagram*

www.EBooksWorld.ir

Solution Explorer View

Figure 16-2 shows the high-level structure of the parts of the program.

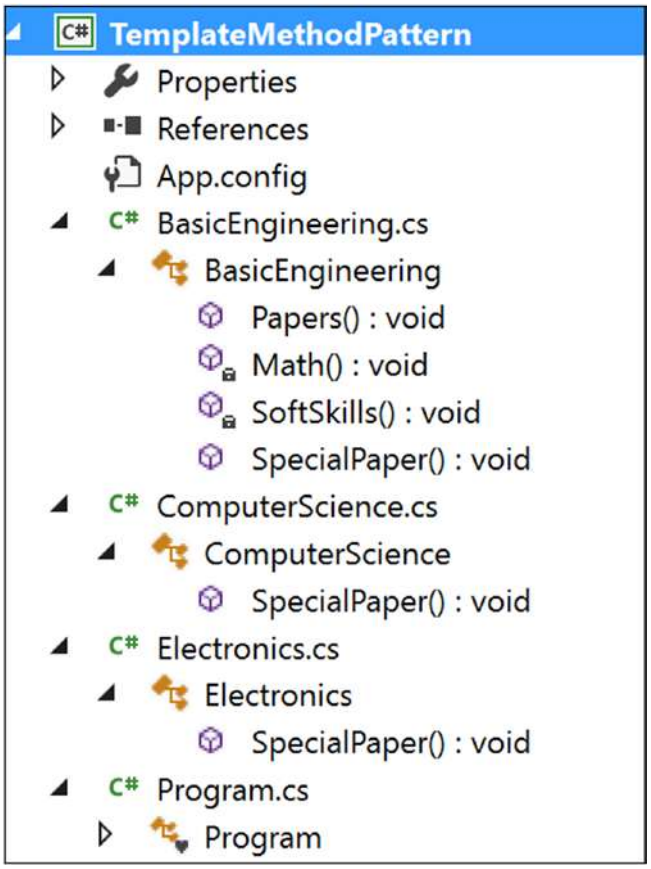


Figure 16-2. *Solution Explorer View*

Implementation

Here's the implementation:

```
// BasicEngineering.cs
using System;

namespace TemplateMethodPattern
{
   public abstract class BasicEngineering
    {
```


www.EBooksWorld.ir

```
        public void Papers()
        {
            //Common Papers:
            Math();
            SoftSkills();
            //Specialized Paper:
            SpecialPaper();
        }
        private void Math()
        {
            Console.WriteLine("Mathematics");
        }
        private void SoftSkills()
        {
            Console.WriteLine("SoftSkills");
        }
        public abstract void SpecialPaper();
    }
}

//ComputerScience.cs
using System;

namespace TemplateMethodPattern
{
    public class ComputerScience:BasicEngineering
    {
       public override void SpecialPaper()
        {
            Console.WriteLine("Object-Oriented Programming");
        }
    }
}
```


www.EBooksWorld.ir

```
//Electronics.cs

using System;

namespace TemplateMethodPattern
{
    public class Electronics:BasicEngineering
    {
        public override void SpecialPaper()
        {
            Console.WriteLine("Digital Logic and Circuit Theory");
        }
    }
}

// Program.cs
using System;

namespace TemplateMethodPattern
{
    class Program
    {
        static void Main(string[] args)
        {

Console.WriteLine("***Template Method Pattern Demo***\n");
            BasicEngineering bs = new ComputerScience();
                        Console.WriteLine("Computer Sc  Papers:");
            bs.Papers();
            Console.WriteLine();
            bs = new Electronics();
            Console.WriteLine("Electronics Papers:");
            bs.Papers();
            Console.ReadLine();
        }
    }
}
```


www.EBooksWorld.ir

Output

Here's the output:

```
***Template Method Pattern Demo***

Computer Sc  Papers:
Mathematics
SoftSkills
Object-Oriented Programming

Electronics Papers:
Mathematics
SoftSkills
Digital Logic and Circuit Theory
```

Q&A Session

1. **In this pattern, subclasses can simply redefine the methods based on their needs. Is this understanding correct?**

   Answer:

   Yes.

2. **In the abstract class `BasicEngineering`, only one method is abstract, and the other two methods are concrete methods. What is the reason behind this?**

   Answer:

   This is a simple example with only three methods, and you want the subclasses to override only the `SpecialPaper()` method here. Other methods are common to both courses, and they do not need to be overridden by the subclasses.

www.EBooksWorld.ir

3. **Suppose you want to add some more methods in the `BasicEngineering` class, but you want to work on those methods if and only if your child classes need them. Otherwise, you will ignore them. This type of situation is common in some PhD programs where some courses are mandatory, but if a student has certain qualifications, the student may not need to attend the lectures for those subjects. Can you design this kind of situation with the Template Method pattern?**

   Answer:

   Yes, you can. Basically, you want to use a hook, which is a method that can help you to control some kind of flow in an algorithm.

   To show an example of this kind of design, I am adding one more method in `BasicEngineering` called `AdditionalPapers()`. Let's assume that computer science students need to complete this mandatory course, but electronics students can opt out. Let's go through the program and output.

Modified Implementation

```
// BasicEngineering.cs [Changes are in bold letters]
using System;

namespace TemplateMethodPatternQAs
{
   public abstract class BasicEngineering
    {
        public void Papers()
        {
            //Common Papers:
            Math();
            SoftSkills();
            if (IsAdditionalPapersNeeded())
            {
                AdditionalPapers();
            }
```


www.EBooksWorld.ir

```
            //Specialized Paper:
            SpecialPaper();
        }
        private void Math()
        {
            Console.WriteLine("Mathematics");
        }
        private void SoftSkills()
        {
            Console.WriteLine("SoftSkills");
        }
        private void AdditionalPapers()
        {
            Console.WriteLine("AdditionalPapers are needed in this stream.");
        }
        public abstract void SpecialPaper();
        //A hook method-Additional Papers not needed.
        public virtual bool IsAdditionalPapersNeeded()
        {
            return true;
        }
    }
}

//ComputerScience.cs [No Change]

using System;

namespace TemplateMethodPatternQAs
{
    public class ComputerScience:BasicEngineering
    {
       public override void SpecialPaper()
        {
            Console.WriteLine("Object-Oriented Programming");
        }
```


www.EBooksWorld.ir

```
        //Not tested the hook method:
        //Additional papers are needed
    }
}
//Electronics.cs [Changes are in bold letters]
using System;

namespace TemplateMethodPatternQAs
{
    public class Electronics:BasicEngineering
    {
        public override void SpecialPaper()
        {
            Console.WriteLine("Digital Logic and Circuit Theory");
        }
        //Using the hook method
        public override bool IsAdditionalPapersNeeded()
        {
            return false;
        }
    }
}
//Program.cs [No Change]
using System;

namespace TemplateMethodPatternQAs
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("***Template Method Pattern QAs***\n");
            BasicEngineering bs = new ComputerScience();
            Console.WriteLine("Computer Sc  Papers:");
            bs.Papers();
            Console.WriteLine();
            bs = new Electronics();
```


www.EBooksWorld.ir

```
            Console.WriteLine("Electronics Papers:");
            bs.Papers();
            Console.ReadLine();
        }
    }
}
```

Modified Output

Here's the modified output:

```
***Template Method Pattern QAs***

Computer Sc  Papers:
Mathematics
SoftSkills
AdditionalPapers are needed in this stream.
Object Oriented Programming

Electronics Papers:
Mathematics
SoftSkills
Digital Logic and Circuit Theory
```

Note You may prefer an alternative approach. For example, you could override the method in `Electronics.cs`, and then you could make the method body empty. But this approach does not work well if you compare it to the previous approach.

---

4. **It looks like this pattern is similar to the Builder pattern. Is this understanding correct?**

   Answer:

   No. Don't forget the core intent; the Template Method pattern is a behavioral design pattern, and Builder is a creational design pattern. In the Builder pattern, the clients/customers are the boss. They can control the order of the algorithm. In the Template

www.EBooksWorld.ir

Method pattern, you are the boss. You put your code in a central location (for example, the abstract class `BasicEngineering.cs` in this example), and you have absolute control over the flow of the execution. You can alter your template as per your needs, and other participants need to follow you.

5. **What are the key advantages of using a Template Method design pattern?**

   Answer:

   - You can control the flow of the algorithms. Clients cannot change them.
   - Common operations will be in a centralized location. For example, in an abstract class, the subclasses can redefine only the varying parts so that you can avoid redundant code.

6. **What are the key challenges associated with a Template Method design pattern?**

   Answer:

   - The client code cannot direct the sequence of steps. If you want that type of functionality, use the Builder pattern.
   - A subclass can override a method defined in the parent class (in other words, hiding the original definition in the parent class), which can go against the Liskov substitution principle that basically says that if S is a subtype of T, then objects of type T can be replaced with objects of type S. You can learn more here: https://en.wikipedia.org/wiki/Liskov_substitution_principle.
   - Having more subclasses means more scattered code, which is difficult to maintain.

www.EBooksWorld.ir